

的体现就是时间片轮转（Round Robin，RR）策略^[1]。图 7.7 展示了三个任务 A、B、C 在 RR 策略下的执行流。该 RR 策略使用 1 秒的时间片，以 3 秒为一个循环执行这三个任务。由于时间片一般会设为足够小，所以所有任务都可以在一定时间内执行并响应用户，从而将响应时间限定在一个可接受的范围内。相比于 SJF 策略和 STCF 策略，RR 策略不仅无须预知任务的运行时间，而且也不会出现长任务饥饿的情况。

对于 RR 策略，一个关键点是时间片大小该如何选取。理想情况下，时间片选取得越小，任务的响应时间就更短。但是，之前的讨论是基于抽象模型进行的，没有考虑调度器的调度开销和任务切换所导致的上下文切换开销。在实际场景中，过小的时间片反而会引入大量开销，使任务的调度成为严重的性能瓶颈。另一方面，如果时间片过长，比如 100 秒，RR 策略不仅无法满足用户对于响应时间的需求，还很可能产生与 FCFS 策略相同的负面效果。因此，如何将时间片降低到一个合理的值对于开发者是一大挑战，它要求开发者对整个应用场景有着明确的认知。

弊端：任务运行时间相似的场景下平均周转时间高

本节主要考虑的调度指标是响应时间，因此 RR 策略理应有着很好的表现。那么对于周转时间这一指标呢？再以图 7.7 的任务 A、B、C 为例，假设它们的运行时间都是 3 秒，那么它们会分别会在第 7 秒、第 8 秒、第 9 秒完成，这个执行流代表每个任务都在最后时刻完成，它们的平均周转时间非常高。RR 策略在一定程度上保证了每个任务之间的公平性（每个任务都能在一定的时间被调度），同时也获得良好的响应时间。但是，在特定场景（如每个任务运行时间相似）下，任务的平均周转时间可能较差。

总体上，RR 策略是一个非常通用的策略，在批处理任务和交互式任务混合场景下有相对较好的综合表现，且无须预知任务执行时间。但 RR 策略不适用于任务运行时间相似且对平均周转时间有极高要求的场景。

7.3.5 经典调度策略的比较

本节介绍了经典的 FCFS、SJF、STCF 和 RR 调度策略，对这些策略的总结如表 7.5 所示。

表 7.5 经典调度策略的总结

	优 势	弊 端
先到先得（FCFS）	设计实现简单	长短任务混合场景下对短任务不友好
	对所有任务公平	对 I/O 密集型任务不友好
最短任务优先（SJF）	对短任务友好	必须预知任务运行时间
		性能表现严重依赖任务到达时间点
		对长任务不公平

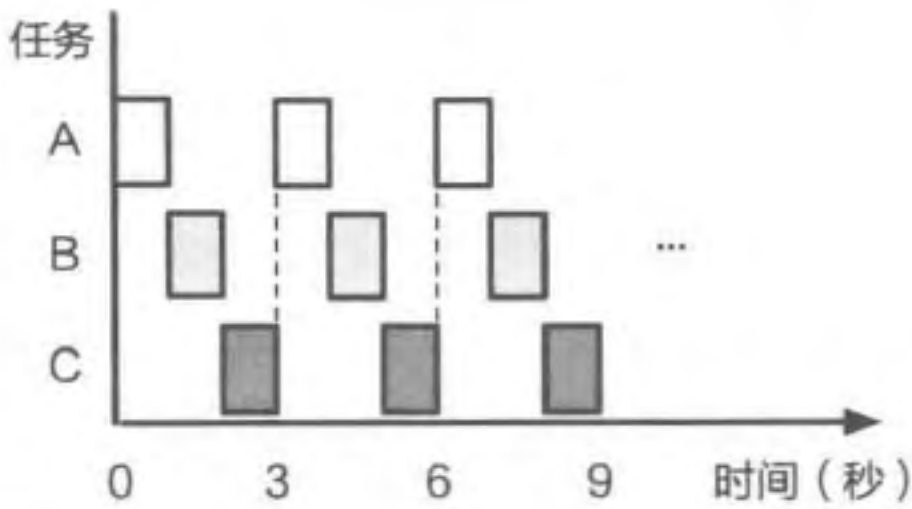


图 7.7 RR 策略在任务运行时间相似场景下的执行流

(续)

	优势	弊端
最短完成时间优先 (STCF)	对短任务友好	必须预知任务运行时间
	支持抢占	对长任务不公平
时间片轮转 (RR)	任务平均响应时间低	任务运行时间相似的场景下平均周转时间高
	对所有任务公平	

7.4 优先级调度策略

本节主要知识点

- ❑ 优先级与不同调度策略的关系是什么？
- ❑ 有哪些典型的优先级调度？
- ❑ 如何设置任务的优先级？

在了解了经典调度策略后，本节将介绍应用广泛的优先级调度。现假设以下场景：小明在一个单核计算机上通过多个后台程序处理大数据集，这些后台程序会在指定的文件目录中输出中间结果文件。小明希望通过命令行的 `ls` 指令看到数据处理的进度。我们假设系统使用 `RR` 策略进行调度，那么现在系统中有 N 个批处理任务和一个命令行的交互式任务。基于 `RR` 策略的调度不会对管理的任务进行区分，因此单位时间内命令行只会占用 $1/(N+1)$ 的 CPU 时间。大部分时间下，后台程序会阻塞 `ls` 指令的执行，小明会感觉整个系统很卡顿，就连 `ls` 都需要很长时间才能响应。如果小明脾气不好，他很有可能马上通过 `kill` 指令终止后台程序。因此，为了保持良好的用户体验，调度器应尽量避免交互式任务被批处理任务阻塞。

如果操作系统有能力让交互式任务优先于批处理任务执行，就能避免交互式任务被批处理任务阻塞。为此，调度器引入了优先级的概念。优先级是一个直接有效的概念，通过为每个任务指定优先级，调度器可以确认哪些任务应该优先执行。只要操作系统可以分辨一个任务是交互式任务还是批处理任务^①，就能为交互式任务设置更高的优先级，进而为用户提供更好的体验。

其实，之前讨论的算法都隐式地使用了优先级这一概念。表 7.6 给出了 7.3 节讨论的调度策略以及对应策略确定任务优先级的方式。可以看到，虽然我们讨

表 7.6 不同调度策略对应的优先级确定方式

调度策略	优先级确定方式
FCFS	任务等待时间长
SJF	任务运行时间短
STCF	任务剩余完成时间短
RR	所有任务平等

① 操作系统可以通过让程序员显式指定的方式或通过启发式算法自动判断的方式判断一个任务的种类。

论了很多不同算法，但是它们最主要的不同点在于如何确定一个任务的优先级。在理解优先级与调度的关系后，相信读者也能够从更高的层次来理解调度。

7.4.1 高响应比优先

现在，我们回顾 FCFS 策略和 SJF 策略以及它们的优缺点。FCFS 策略体现了“先到先得”的原则，先到达的任务优先调度。换句话说，FCFS 策略更倾向于先执行等待时间较长的任务。不可避免地，FCFS 策略对于短任务很不友好，会导致短任务因被长任务阻塞而等待过长时间。为了让运行时间短的任务可能尽快执行，SJF 策略将任务的运行时间作为优先级来调度任务。然而，这也使得运行时间长的任务因陷入饥饿而无法执行。

那么，是否可以设计一种调度策略，既可以让短任务优先执行，又不会让长任务等待太久呢？答案是可以，这种调度策略的设计思路是，让优先级的设置同时考虑任务的运行时间和任务的等待时间。根据上述思路设计的调度策略就是高响应比优先（Highest Response Ratio Next, HRRN）策略。

响应比（Response Ratio）反映了一个任务的等待时间与其运行时间的相对关系，计算公式如下：

$$\text{响应比} = \frac{T_{\text{响应}}}{T_{\text{运行}}} = \frac{T_{\text{等待}} + T_{\text{运行}}}{T_{\text{运行}}}$$

具体地，响应比是一个任务的响应时间与其运行时间的比值，该比值的最小值为 1，表示任务未经过等待就可以立即执行至结束。如果两个任务等待了相同的时间，则运行时间越短的任务响应比越高。如果一个任务的运行时间固定，则等待时间越长，响应比越高。

从响应比的数学意义上理解，“高响应比优先”同时隐含了“优先调度短任务”和“优先调度等待时间长的任务”的思想。观察转换后的响应比公式：

$$\text{响应比} = \frac{T_{\text{等待}}}{T_{\text{运行}}} + 1$$

不难发现，在比较两个任务的响应比时，两边的常数项 1 会被抵消。实际上，比较两个任务的响应比，就是在比较它们的等待时间与运行时间之比，这也是 HRRN 策略选取响应比作为优先级的本质原因。

总体上，HRRN 策略通过结合 FCFS 策略和 SJF 策略，避免了 SJF 策略与 STCF 策略在公平性方面的问题。对于大部分场景，只需预知任务执行时间就能有不错的平均周转时间和吞吐量。

7.4.2 多级队列与多级反馈队列

在了解了 HRRN 策略后，读者不难发现，并非只有单一因素能够影响任务的优先

级。实际上，真实系统中为了确定任务优先级需要考虑很多因素，情况非常复杂。一种直接的设置优先级的方式是人为划分任务的种类，根据不同类任务在系统中的重要程度分配优先级。那么作为开发者，应该如何为不同的任务安排优先级呢？对于拥有明确截止时间的实时任务，我们应该为其分配最高优先级，尽量保证实时任务可以在截止时间以前完成；而交互式任务的响应时间直接影响用户体验，为了避免用户体验较差，我们需要为其分配较高的优先级；批处理任务一般没有如实时任务与交互式任务那样对时延、响应时间的高要求，所以这类任务的优先级较低。本节将首先介绍一种直观的静态优先级调度策略——多级队列，基于该策略的任务会严格根据预设的优先级进行调度。基于对该调度策略的观察，本节会逐步给出一些关于设置优先级的经验。那么，针对完全未知的任务，该如何设置优先级并保证系统整体较优的性能呢？本节会介绍一个经典的基于多级队列的动态优先级调度策略——多级反馈队列。

多级队列

我们试想一个真实的生活场景。在机场有一架航班准备起飞，我们将需要登机的人分为三类：①飞行员、空乘、维护人员；②购买商务舱机票的VIP乘客；③其他普通乘客。在飞机允许登机以前，第一类工作人员会在专用场所等待，VIP乘客一般会在VIP候机室等待，其他乘客则会在候机大厅等待。在飞机到达后，工作人员需要立即登机进行维护、检查并完成临起飞工作，可以认为他们的登机优先级最高。当工作人员准备好以后，后两类乘客可以开始登机。而在登机时，一般是VIP乘客全部登机完，再请普通乘客登机。我们将这个场景与处理器调度进行类比：工作人员就是实时任务，他们负责在规定时间内确保飞机可以按时起飞；VIP乘客与交互式任务一样有着相对较高的优先级；而其他乘客则与批处理任务类似，其优先级低于实时任务和交互式任务。这三类人员被分在不同的三个场所等待，就好像操作系统将不同优先级的任务放在不同的任务队列中。另外，对于优先级相同的人员，对他们的登机顺序并没有明确的要求：工作人员只需要比乘客先行登机，按时做好准备工作即可；另外两类乘客一般采用排队即先到先得的方式来确定登机顺序。

多级队列（Multi-Level Queue, MLQ）策略就是以类似的思想进行设计的，如图7.8所示。每个任务会被分配预先设置好的优先级，每个优先级对应一个队列，任务会被存储在对应的优先级队列中。如果优先级不同的任务同时处于就绪状态，那么调度器应该倾向于调度优先级较高的任务，因此一个任务必须等到所有优先级比它高的任务被调度完才可以被调度。处于相同优先级队列的任务，对它们的调度方式没有统一标准，可以有针对性地为不同队列采用不同调度策略，例如FCFS策略^①或RR策略。

MLQ适合静态的应用场景，这类场景下的任务信息（例如任务的大致运行时间、资源使用情况等）可以在执行前获知。根据任务信息，可以生成对应的调度模型，进而计

① 此处的FCFS策略是可以被更高优先级任务抢占的。

算出每种任务适合的优先级并进行调度。

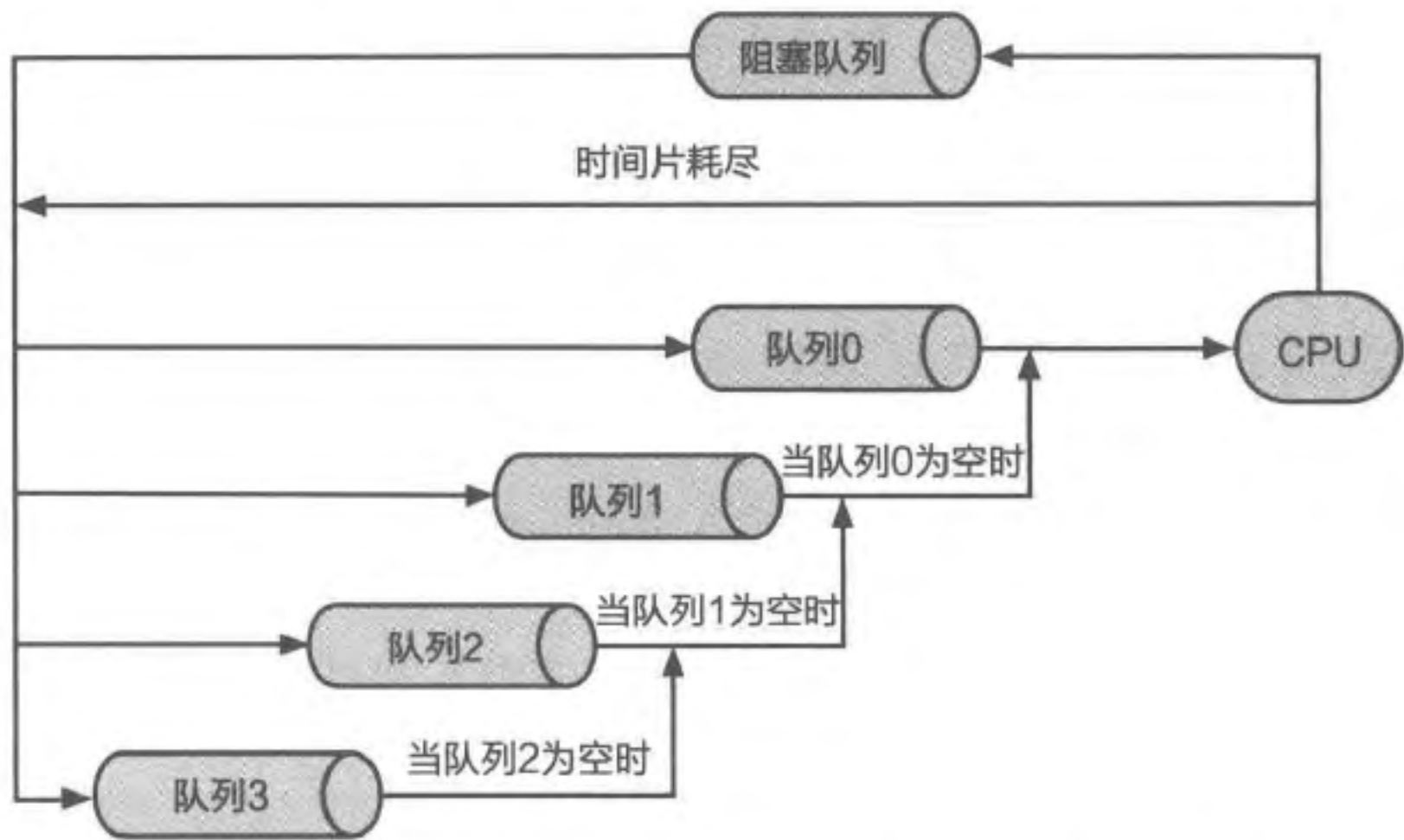


图 7.8 优先级调度策略示意图（队列 0 对应最高优先级）

在设置 MLQ 的任务优先级时，需要注意将 I/O 密集型任务的优先级提高。试想一种计算密集型任务与 I/O 密集型任务混合运行的场景，假设计算密集型任务的优先级比 I/O 密集型任务的优先级高。那么就很可能遇到以下情况：大量高优先级的计算密集型任务优先于 I/O 密集型任务执行，而 I/O 密集型任务一直没有机会发起 I/O 请求，必须等到所有计算密集型任务做完才可以执行，最终造成 I/O 资源利用率低下。由于 I/O 密集型任务一般不会消耗大量 CPU 资源，完全可以将其优先级提高，率先发起 I/O 请求，从而充分利用空闲的 I/O 资源。

MLQ 是一种高效的优先级调度策略，由于队列的数量一般是预先确定的，调度器可以在 $O(1)$ 时间（相对于任务总量 N ）找到所有非空队列中优先级最高的一个，并选取其队列头的任务进行调度。同时，MLQ 可以如实地反映任务的优先级。

然而，完全基于预设的优先级调度任务也可能带来问题：低优先级任务饥饿。如图 7.9 所示，在调度时很有可能出现一个低优先级的任务等待执行，但是大量高优先级的任务不断进入系统，造成低优先级的任务无法被调度，即低优先级任务饥饿。如果调度器需要保证一定的公平性，避免 MLQ 引起的饥饿，则需要额外的机制来监控任务等待时间，并为等待时间过长（如超过一定阈值）的任务提升优先级。

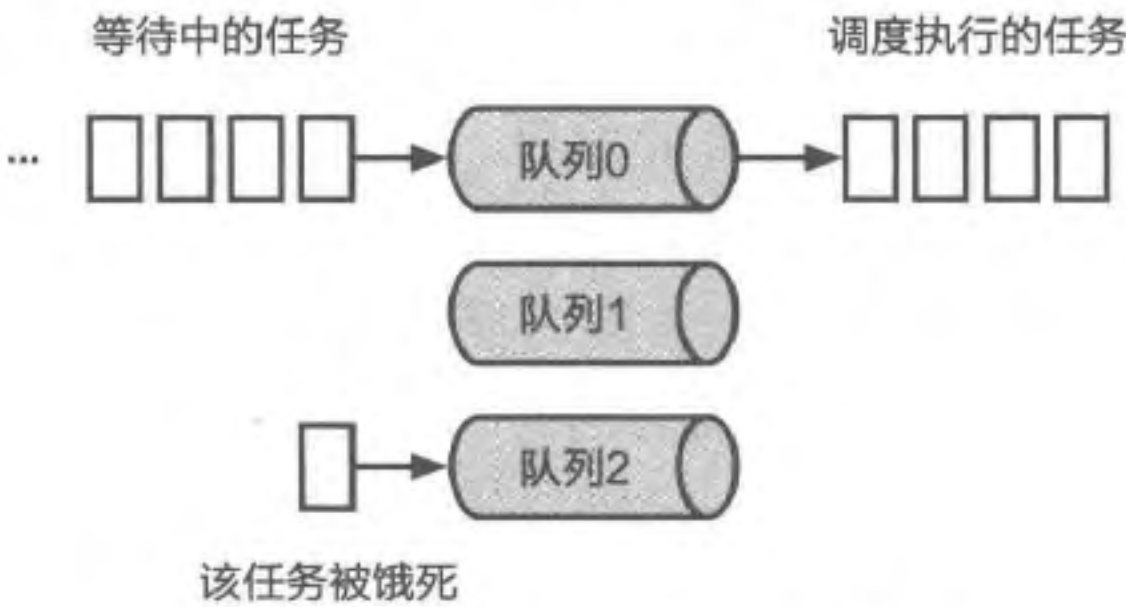


图 7.9 低优先级任务饥饿示意图

小思考

在了解了什么是多级队列后，细心的读者会隐约发现，多级队列更像是一种优先级调度的实现方式或机制。实际上，这种直觉是有一定道理的。前文讨论的各种调度策略更多的是在讨论任务的优先级如何设置，而非如多级队列一样去设计一种数据结构来定位优先级最高的任务。但另一方面，我们也可以将多级队列视为优先级范围被静态确定的优先级调度，那么它也可以被视为一种调度策略。

对于多级队列到底是一种调度策略还是调度机制这一问题，可能没有标准答案。这个小思考是希望大家能够融会贯通，有更多自己的思考。

小思考

试想一种直观的优先级调度实现：所有任务存储于同一数据结构（与多级队列将任务根据优先级存储于不同数据结构相对），每次调度选取优先级最高的任务。多级队列与之相比的优劣分别是什么？

多级队列的缺点是限制了任务的优先级，即任务的优先级只能在给定的优先级范围内，而上述方法则可以为任务分配任意优先级。也正是由于优先级范围的限制，多级队列的优点是可以减小调度开销。具体原因如下。一般为了维护所有任务中优先级最高的任务信息，需要遍历所有任务，其时间复杂度为 $O(N)$ ， N 对应需要处理的任务数量。从算法的角度考虑，维护一个有序的堆或树状数据结构时，虽然该数据结构的查询（找到优先级最高的任务）时间复杂度为 $O(1)$ ，但维护（将一个任务放回运行队列）时间复杂度仍为 $O(\log N)$ 。而多级队列相当于维护了一个数组，任务可以直接根据优先级找到对应的队列，查询、维护的时间复杂度都为 $O(1)$ 。并且，一般只要多级队列所设定的优先级范围足够大，其优先级范围受限的缺点也是可以忽略的。

多级反馈队列

随着计算机的发展，操作系统中任务的需求变得越来越复杂，任务可能同时希望有较低的周转时间与响应时间，并且任务的运行时间无法预知。前文中介绍的 STCF 策略和 RR 策略都无法同时满足上述需求。因此，寻找一个综合全面的、适应动态应用场景的调度策略，成为当时调度研究的目标之一。图灵奖获得者 Corbató 领导相容分时系统（Compatible Time-Sharing System, CTSS）的研发，并于 1962 年提出多级反馈队列^[2]。多级反馈队列（Multi-level Feedback Queue, MLFQ）策略的设计目标是，在任务信息无法预知且任务类型动态变化（任务行为模式会在计算密集型与 I/O 密集型之间转换）的

场景下，既能达到类似 STCF 策略的周转时间，又像 RR 策略一样可以尽可能降低任务的响应时间。为此，MLFQ 策略在沿用 MLQ 的基础上，增加了动态设置任务优先级的策略。

与 MLQ 类似，MLFQ 策略也会维护多个优先级队列，任务根据优先级存于不同队列中，高优先级任务先于低优先级任务执行，处于相同优先级的任务则采用 RR 策略执行。MLFQ 策略的一大创新是实现了优先级的动态设置，具体策略如下。

短任务拥有更高的优先级。MLFQ 策略倾向于为短任务设置更高的优先级，主要有以下三点好处。第一，在介绍 SJF 策略时已经提到，优先调度短任务可以获得更好的平均周转时间。第二，I/O 密集型任务一般在 CPU 中执行的时间很短，给短任务提高优先级也相当于提高 I/O 密集型任务的优先级，有利于提高系统的 I/O 资源利用率。第三，交互式任务一般是短任务，提高其优先级有助于降低这些任务的响应时间。

MLFQ 策略首先对任务的运行时间进行评估，预计运行时间较短的任务会放入优先级较高的队列中。但是，在真实系统中，可能无法准确追踪任务的完成时间和剩余完成时间，这也是 SJF 策略和 STCF 策略的一大限制。为此，MLFQ 会统计每个任务已经执行了多长时间，并据此判断该任务是短任务还是长任务。首先，任务进入系统时，MLFQ 会假设该任务是短任务，并将其设置为最高优先级，这有利于交互式任务达成较短的响应时间以及 I/O 密集型任务充分利用 I/O 资源。然后，MLFQ 会为每个任务级队列设置任务的最大运行时间（任务在当前队列可运行的总时间，而非时间片）。如果任务在当前队列（多次）执行并最终超过了队列允许的最大时间，MLFQ 策略会认为该任务是运行时间较长的任务，进而将该任务的优先级减一。凭借该方法，MLFQ 就可以大致适配任务的优先级，即动态评估任务的运行时间。由于短任务一般都可以在预设的时间片之前完成，因此可以一直保留在优先级较高的队列中；相对地，长任务的优先级则会随着执行次数的增多逐渐降低。

低优先级的任务采用更长的时间片。CTSS 起初运行在 IBM 7090 上^[2]，出于内存保护和简化设计的原因，一次只能将一个用户的任务载入内存，因此任务切换的开销非常大。为了尽量减少任务的调度次数，MLFQ 根据预估的任务执行时间，给予一个合适的调度时间片。优先级越低的任务，其时间片越长。由于 MLFQ 策略支持抢占式调度，所以无须担心低优先级的任务阻塞新进入系统的任务。

定时将所有任务的优先级提升至最高。与 MLQ 类似，上述 MLFQ 策略也有可能造成低优先级任务饥饿。当一个长任务经过一定时间的运行后，它最终会被分配到最低的优先级，并且没有其他机制可以重新提升它的优先级。如果此时一直有短任务需要调度，那么长任务就无法执行，造成饥饿现象。在 MLFQ 策略针对的场景下，一个任务原本可能是计算密集型任务，因而被放入最低优先级队列。但在一定时间后，用户希望与其交互，该任务则变成了一个对响应时间有要求的交互式任务。因此，MLFQ 策略会在一定时间周期后将系统内所有任务的优先级重新提升至最高级，保证不会有任务饥饿。

图 7.10 展示了三个任务 A、B、C 基于 MLFQ 策略的执行流。其中任务 A 是一个长任务，而任务 B 和 C 的运行时间相对较短。在初始阶段，任务 A 和 B 进入系统，并且被分配最高的优先级。当它们在队列 0（最高优先级队列）中执行至最大运行时间后，调度器会降低它们的优先级，移至队列 1。任务 B 在队列 1 执行结束，而任务 A 一直在执行，因此任务 A 的优先级会被进一步降低并移至队列 2。当任务 A 在队列 2 执行到第二个时间片时，任务 C 进入系统，它被调度器分配最高的优先级，抢占任务 A 的执行。过了一定的时间后，到达调度器定时提升任务优先级的时间点，任务 A 和 C 都被移至队列 0 执行。任务 C 随后在队列 0 的第一个时间片结束自己的执行，长任务 A 继续执行，并最终再次回到最低优先级的队列 2。

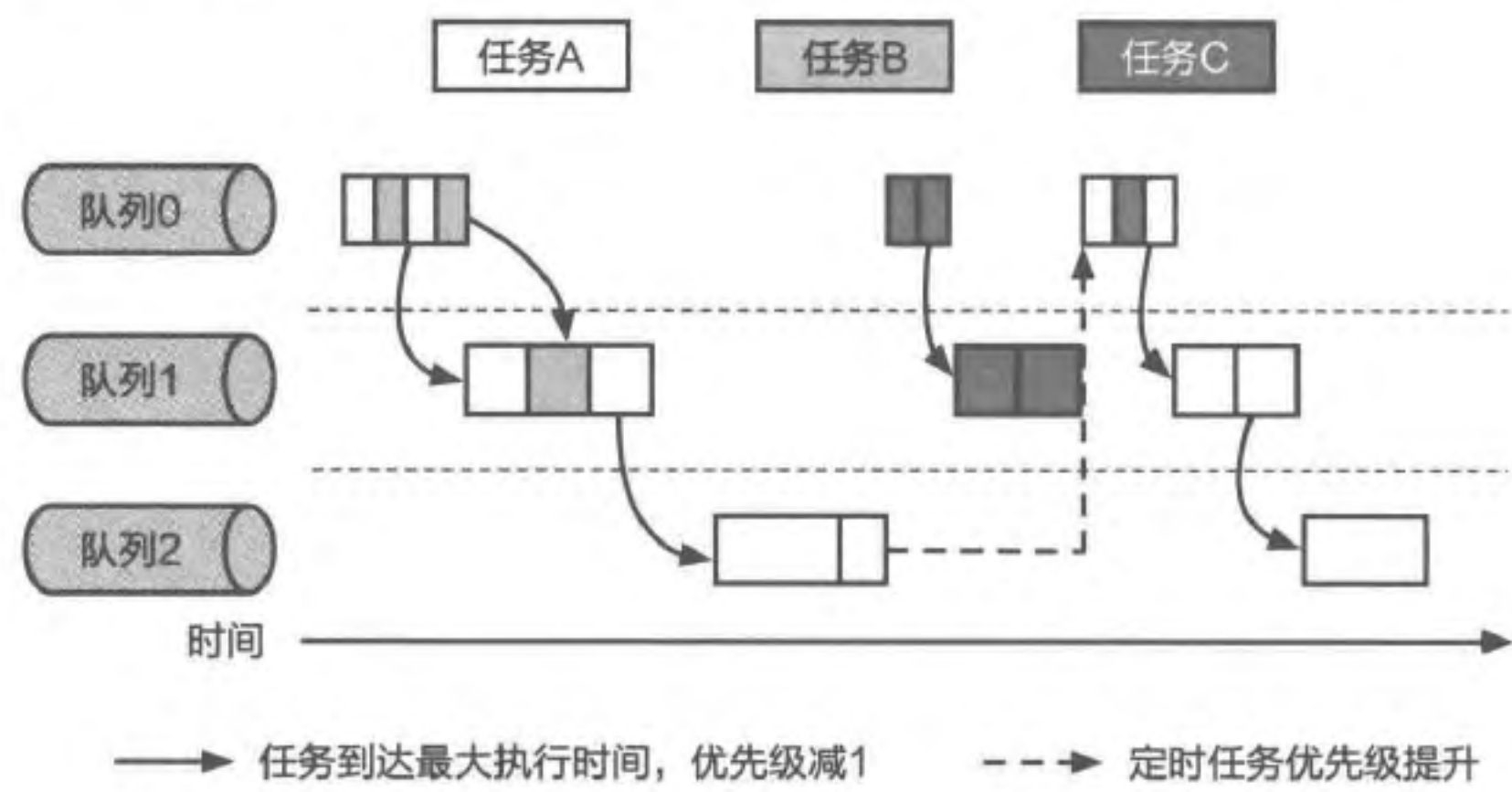


图 7.10 MLFQ 策略的执行流

在 MLFQ 策略的设计中，任务的优先级会被动态地提升（Boost）和降低（Penalty，对应“惩罚”）。提升和降低是用于动态调节任务优先级的机制，被应用在包括 Linux 在内的许多操作系统中。系统可以启发式地确定哪些任务需要被提升或降低优先级，以及优先级的变化量应该是多少，进而调整优先级调度的结果。然而，这些启发式方法在提升调度效果的同时，也可能造成实现复杂的问题。

在 MLFQ 策略的具体实现中，有许多调度参数需要调整，例如优先级队列的数量、每个优先级队列采用的时间片、任务在每个优先级队列的最大运行时间以及调度器定时提升优先级的时间间隔。如果参数使用不当，就可能达不到预期的调度效果。比如，如果提升优先级的时间间隔过短，那么所有任务都会保持在最高优先级的队列中，退化成 RR 策略；而如果该间隔过长，很可能会导致长任务保持在最低优先级队列中，很长时间内无法执行。因此，虽然 MLFQ 调度策略的设计非常巧妙，但开发者仍然需要花费很大的努力，对系统整体有细致的了解，才可以实现出有效的基于 MLFQ 策略的调度器。

MLFQ 策略通过记录任务运行时间的方式，动态地调配任务的优先级。它能达到与 SJF 策略和 STCF 策略类似的低平均周转时间，同时还能保证任务的响应时间，并且避

免了任务饥饿。以 MLFQ 为基础的调度策略在很多操作系统中都得到了应用，例如早期 Linux、Windows 和 macOS^①。

7.4.3 优先级调度策略的比较

本章介绍的 HRRN 策略说明，设置优先级调度策略的优先级是需要考虑多种因素并经过深思熟虑的。在此基础上，本章分别介绍了静态优先级策略 MLQ 和动态优先级策略 MLFQ，展示了实际调度策略如何设置任务的优先级。根据优先级设置策略的不同，它们的区别也显而易见：HRRN 策略主要希望融合 FCFS 与 SJF 策略的优点，在短任务优先的同时保证长任务不会饿死，所以其优先级设置会同时考虑任务的运行时间和等待时间；MLQ 策略主要关注任务已知且种类划分明确的场景，所以其优先级会根据任务种类而设置；而 MLFQ 策略则是一种通用的优先级调度，其不同的规则实际上是为了满足不同的设计考量。因此，在选择优先级调度策略时，读者应该首先明确适合目标场景的特征与调度指标，这样才能设计出合适的优先级调度策略。

7.5 公平共享调度策略

本节主要知识点

- 为什么需要公平共享调度？什么是份额？
- 有哪些典型的公平共享调度策略？
- 份额与优先级的异同是什么？

优先级调度被广泛应用在大量场景中，然而优先级调度也并非在所有场景中都能适用。以资源共享场景为例，当用户共享服务器时，一个默认假设是用户使用的资源数（例如 CPU 时间）与所花费的钱是成正比的。现在，假设小明和小红合租了一台云计算平台上的单核服务器，他们商定平分 CPU 时间，并且小明同时开启三个任务（任务 A、B、C），而小红只开启一个任务（任务 D）。那么有什么调度策略能够保证小明的三个任务和小红的一个任务平分 CPU 时间呢？

假设这些任务的优先级相同，并且采用 RR 策略进行“公平”的调度。那么由于此时小明开启三个任务，会占据 75% 的 CPU 时间，而小红则只能使用剩下的 25%，因此 RR 策略不能满足两人对于 CPU 资源平分的约定。

小明和小红决定尝试优先级调度，小明的三个任务优先级被设置为 1（低优先级），而小红的一个任务优先级被设置为 3（高优先级），这样的优先级分配从数值上体现了用

^① 资料来源于维基百科，参见 [https://en.wikipedia.org/wiki/Scheduling_\(computing\)](https://en.wikipedia.org/wiki/Scheduling_(computing))。